

Object Oriented Programming (3-0-2-4)

Lecture 1

Types of Programming Methodologies

- Unstructured programming
- Procedural/Process oriented programming
 - Modular programming
- Object-oriented programming

Unstructured Programming

- Direct code (historically earliest programming paradigm)
 - Ex: Assembly Language

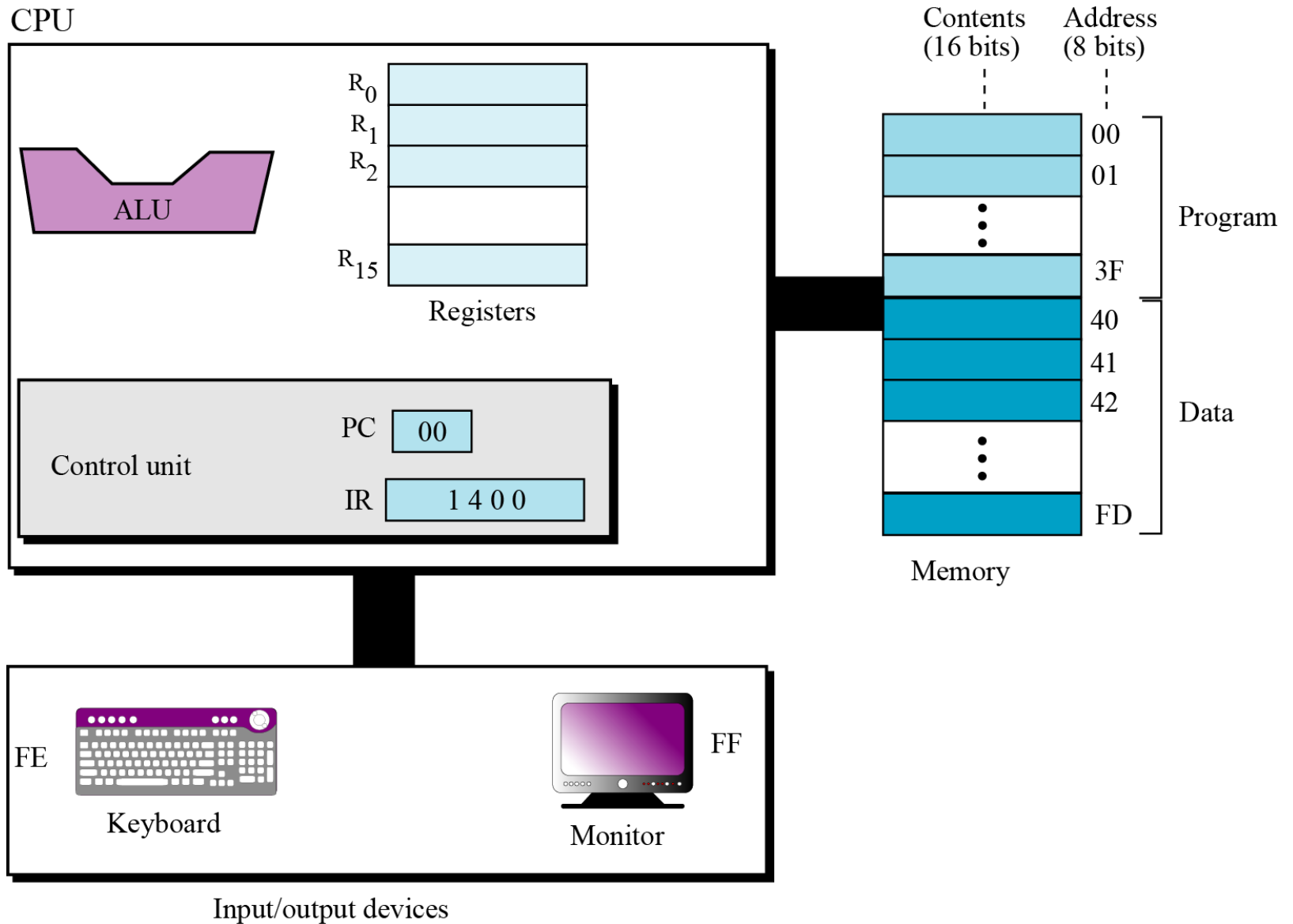


Figure: The components of a simple computer

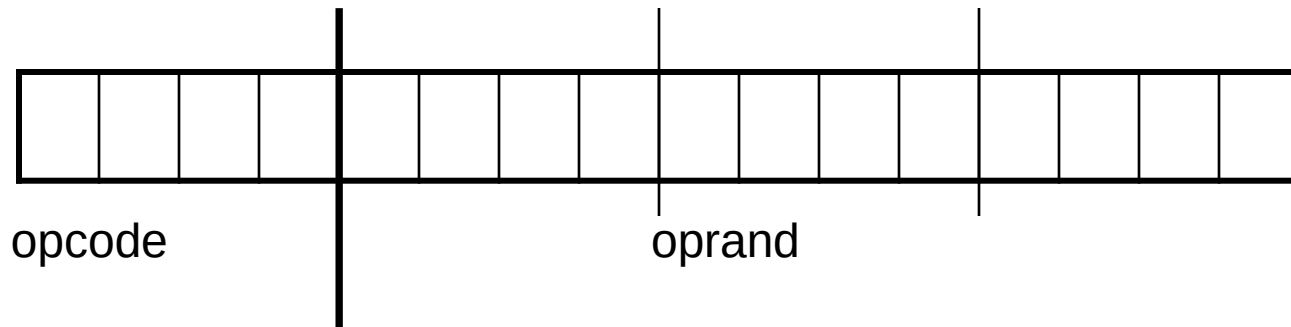
Instruction set

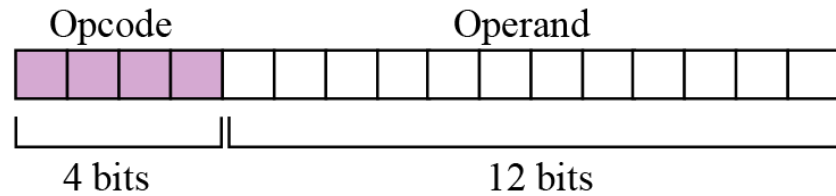
Each computer instruction consists of two parts: the **operation code (opcode)** and the **operand (s)**.

The opcode specifies the type of operation to be performed on the operand (s).

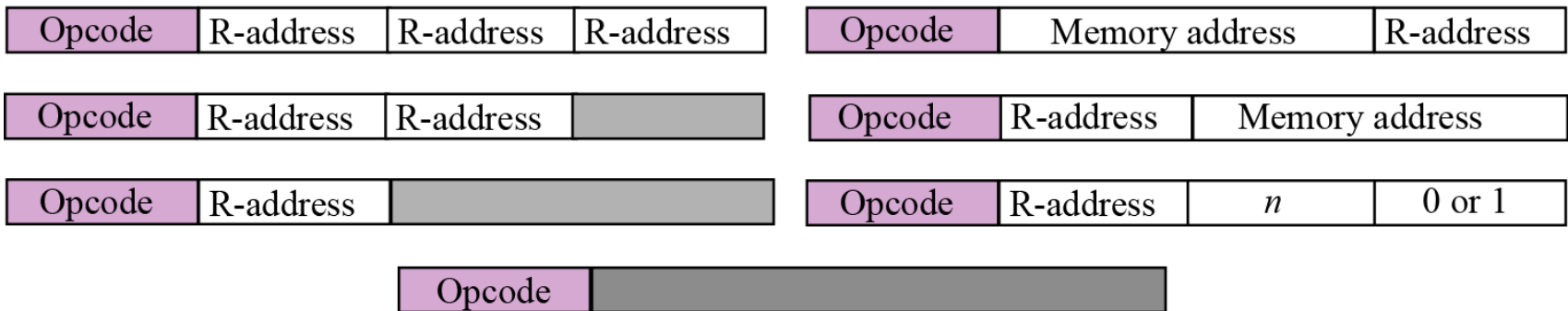
Each instruction consists of sixteen bits divided into four 4-bit fields.

The leftmost field contains the opcode and the other three fields contains the operand or address of operand (s), as shown in next Figure.





a. Instruction format



b. Instruction types

Figure: Format and different instruction types

R-address: address of a register

Memory address: Main memory address

Table List of instructions for the simple computer

Instruction	Code	Operands			Action
	d ₁	d ₂	d ₃	d ₄	
HALT	0				Stops the execution of the program
LOAD	1	R _D	M _S		R _D ← M _S
STORE	2	M _D		R _S	M _D ← R _S
ADDI	3	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} + R _{S2}
ADDF	4	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} + R _{S2}
MOVE	5	R _D	R _S		R _D ← R _S
NOT	6	R _D	R _S		R _D ← $\overline{R_S}$
AND	7	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} AND R _{S2}
OR	8	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} OR R _{S2}
XOR	9	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} XOR R _{S2}
INC	A	R			R ← R + 1
DEC	B	R			R ← R – 1
ROTATE	C	R	n	0 or 1	Rot _n R
JUMP	D	R	n		IF R ₀ ≠ R then PC = n, otherwise continue

Key: R_S, R_{S1}, R_{S2}: Hexadecimal address of source registers
R_D: Hexadecimal address of destination register
M_S: Hexadecimal address of source memory location
M_D: Hexadecimal address of destination memory location
n: hexadecimal number
d₁, d₂, d₃, d₄: First, second, third, and fourth hexadecimal digits

An example

Add two integers A and B and create the result as C.

$$C = A + B$$

We assume that the first two integers are stored in memory locations $(40)_{16}$ and $(41)_{16}$ and the result should be stored in memory location $(42)_{16}$.

To do the simple addition needs five assembly language instructions, as shown next:

1. Load the contents of M_{40} into register R_0 ($R_0 \leftarrow M_{40}$).
2. Load the contents of M_{41} into register R_1 ($R_1 \leftarrow M_{41}$).
3. Add the contents of R_0 and R_1 and place the result in R_2 ($R_2 \leftarrow R_0 + R_1$).
4. Store the contents R_2 in M_{42} ($M_{42} \leftarrow R_2$).
5. Halt.

In the language of our simple computer, these five instructions are encoded as:

<i>Code</i>	<i>Interpretation</i>			
$(1040)_{16}$	1: LOAD	0: R_0	40: M_{40}	
$(1141)_{16}$	1: LOAD	1: R_1	41: M_{41}	
$(3201)_{16}$	3: ADDI	2: R_2	0: R_0	1: R_1
$(2422)_{16}$	2: STORE	42: M_{42}		2: R_2
$(0000)_{16}$	0: HALT			

Processing the instructions

Our simple computer, like most computers, uses machine cycles.

A cycle is made of three phases: **fetch**, **decode** and **execute**.

During the fetch phase, the instruction whose address is in PC is obtained from the memory and loaded into the IR. The PC is then incremented to point to the next instruction.

During the decode phase, the instruction in IR is decoded and the required operands are fetched from the register or from memory.

During the execute phase, the instruction is executed and the results are placed in the appropriate memory location or the register.

Once the third phase is completed, the control unit starts the cycle again, but now the PC is pointing to the next instruction. The process continues until the CPU reaches a HALT instruction.

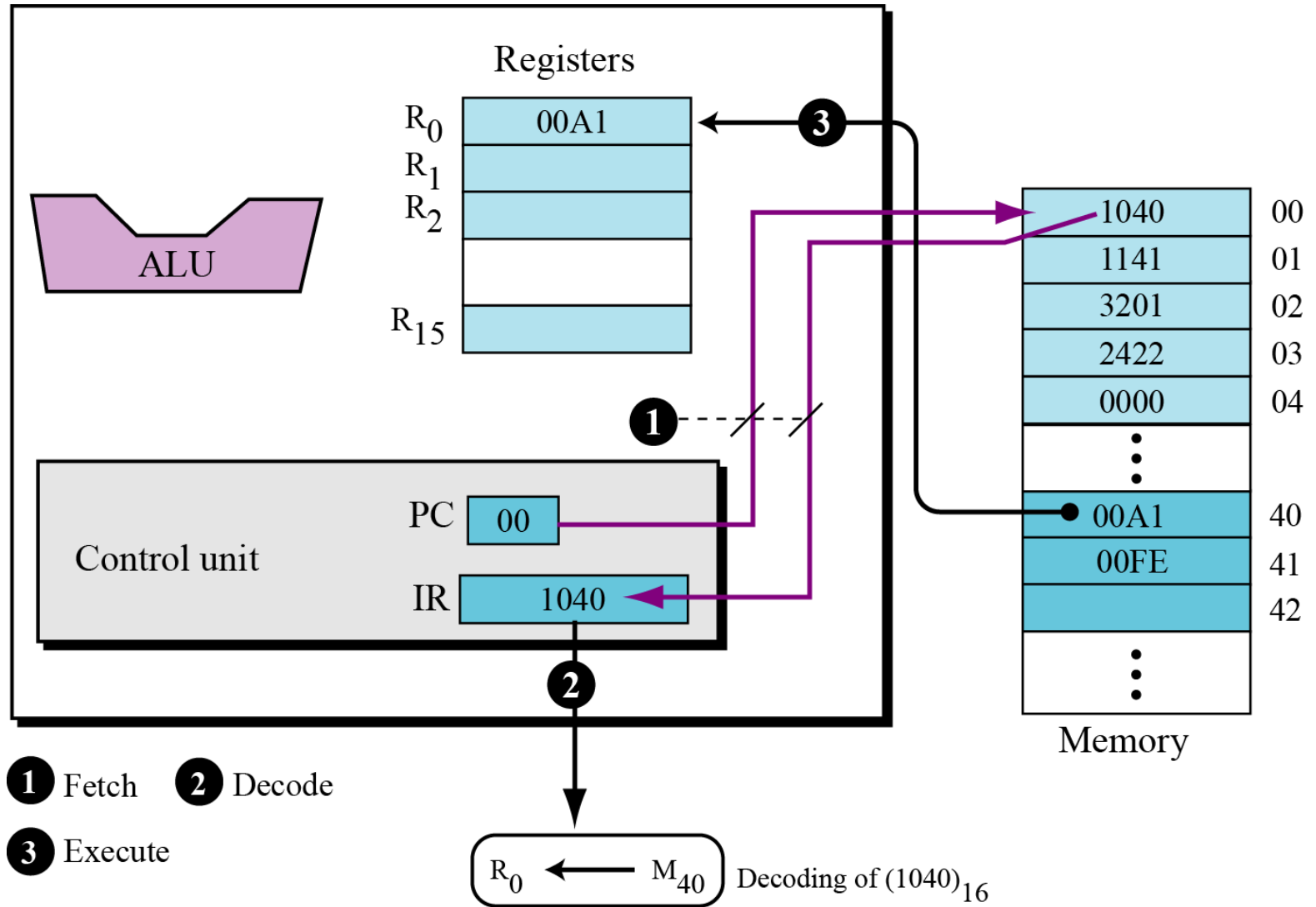


Figure Status of cycle 1

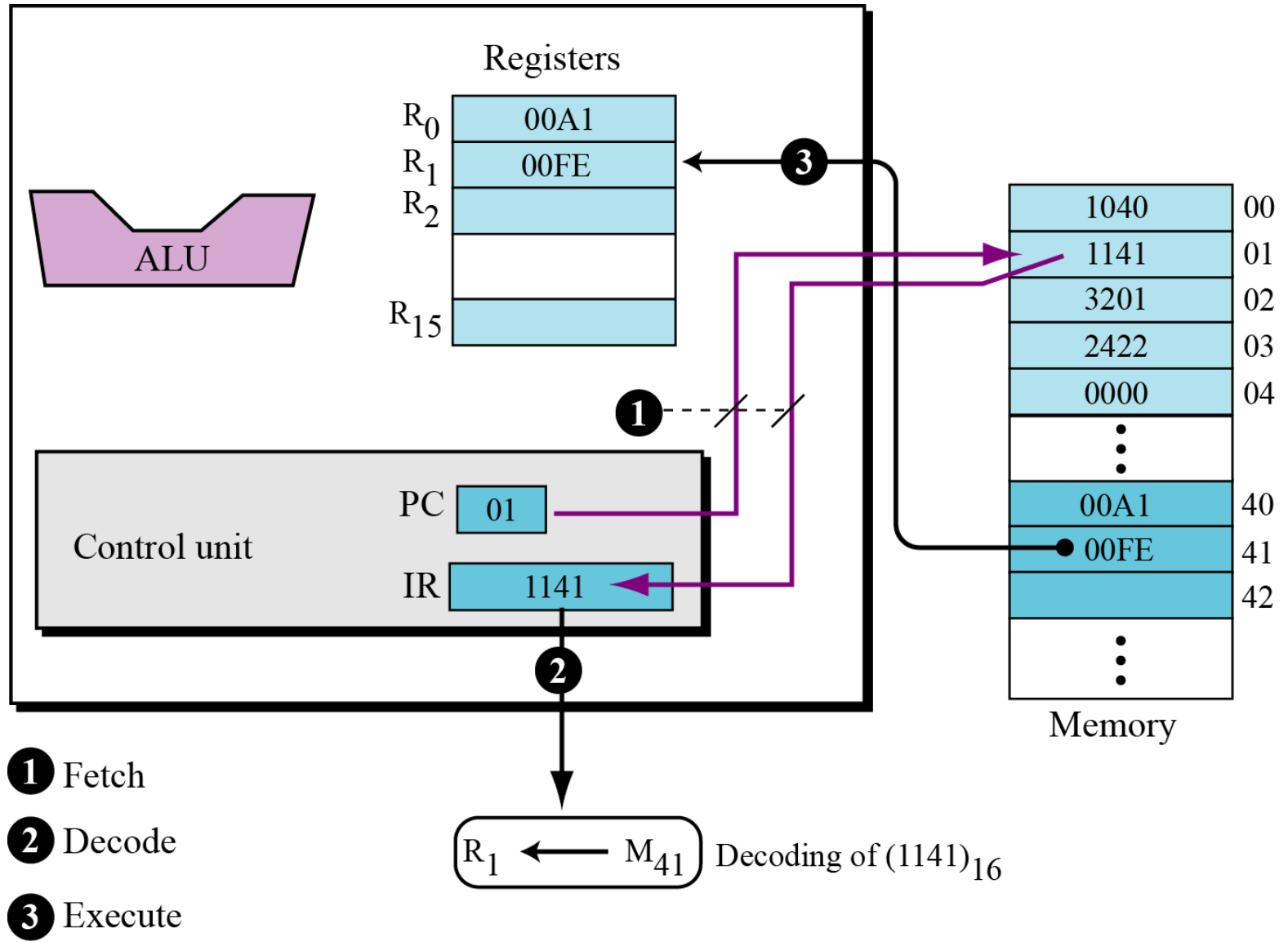


Figure Status of cycle 2

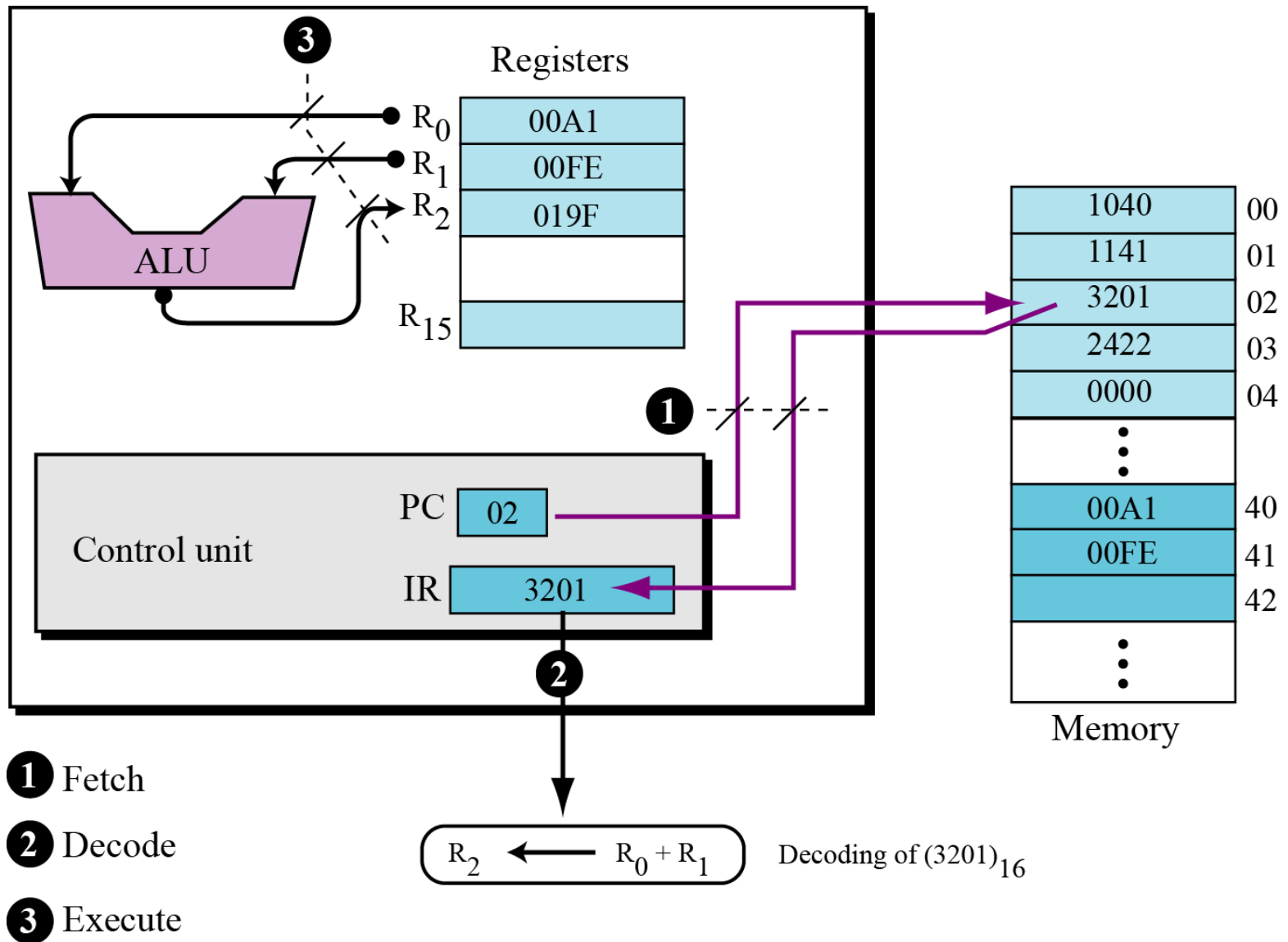


Figure: Status of cycle 3

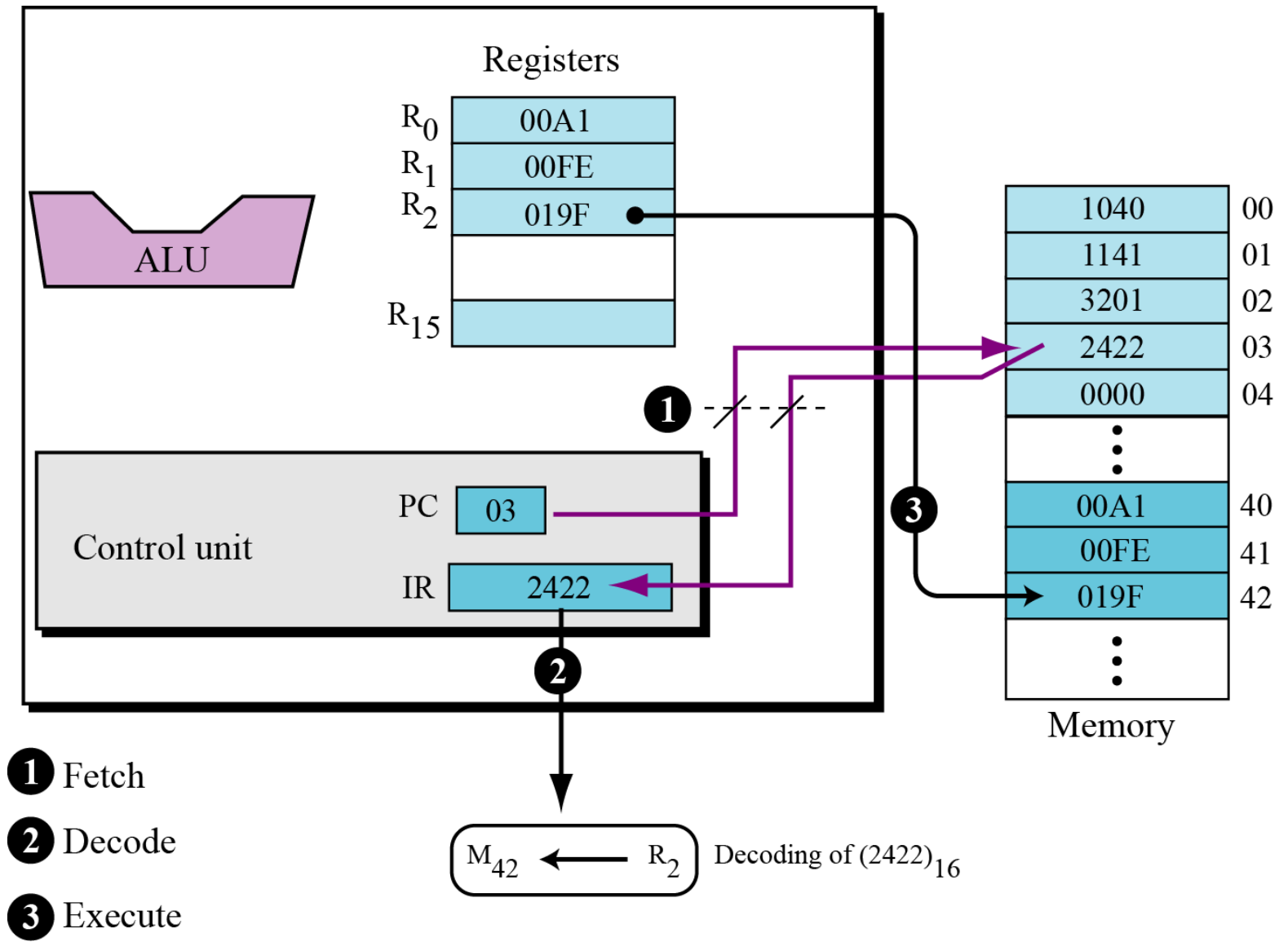


Figure: Status of cycle 4

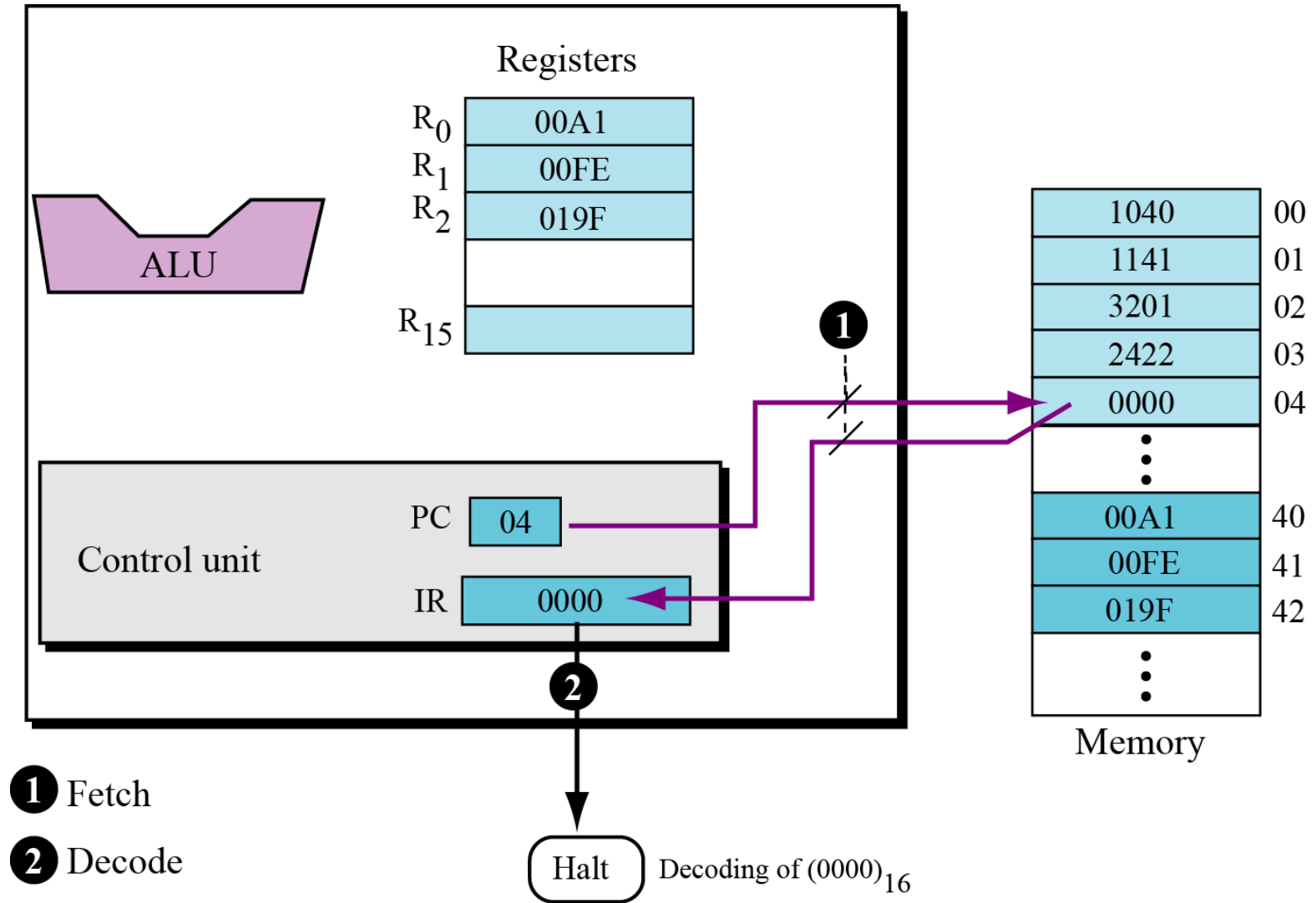


Figure: Status of cycle 5

Unstructured Programming

Limitations

- As code increases, complexity increases
 - Hard to manage large programs
 - Hard to debug
- No re-usability
- Statements written many times